

Assignment 1 Report

Bridging Statistical Learning and Neural Networks for Regression

CSCI933 – Machine Learning Algorithms and Applications

Student Name: Sohan Ramesh Kumar
Student Number: 1093022
Lecturer: Professor Philip O. Ogunbona
Date Submitted: 25th April 2026

1 Introduction

This report looks at predicting bike rental demand using two different approaches: classical statistical regression and neural networks. The question I wanted to answer was whether these two frameworks — which feel quite different in practice — actually arrive at the same place when the problem is simple enough, and where they start to pull apart when things get more complex.

There is a theoretical claim sitting underneath this: a linear neural network trained with L2 weight decay should be mathematically equivalent to ridge regression. That is not an obvious thing to just accept, so part of the work here was to derive it properly and then check it holds in practice.

Regularisation is the other main thread. Ridge, lasso, elastic net, weight decay, dropout — these are all different answers to the same question: how do you stop a model from memorising the training data? Understanding when each one is the right tool, and why, is something I tried to build up through both the derivations and the experiments.

I used GenAI as a thinking aid during this assignment — mostly to get a first pass at the mathematical derivations and to probe claims I was unsure about. I treated every response as something to question rather than accept, and the places where the AI fell short turned out to be the most useful parts of the process. That interaction is documented in Appendix A.

2 Dataset and Preprocessing

2.1 Dataset Description

The dataset used is the Bike Sharing Dataset [1] from the UCI Machine Learning Repository. It records hourly bike rental counts from Washington D.C.'s Capital Bikeshare system across 2011 and 2012, giving 17,379 observations in total.

Seven features were used as specified in the assignment, with total hourly rentals as the target:

Table 1: Required feature subset

Feature	Description
temp	Normalised temperature in Celsius
humidity	Normalised humidity
windspeed	Normalised wind speed
season	Season (1=winter, 2=spring, 3=summer, 4=autumn)
holiday	Binary: 1 if public holiday
workingday	Binary: 1 if working day
hour	Hour of day (0–23)
bikes_rented	Target: total hourly bike rentals

day.csv was left out — it is just a rolled-up version of the same data with no additional information, and it does not contain the `hour` feature at all.

2.2 Exploratory Analysis

The target ranges from 1 to 977 with a mean of 189 and a fairly right-skewed distribution. Looking at the correlations, temperature ($r = 0.405$) and hour ($r = 0.394$) came out as the strongest predictors. Humidity showed a moderate negative association ($r = -0.323$), while windspeed (0.093), holiday (0.031), and workingday (0.030) were much weaker.

The clearest pattern in the data is the daily demand cycle: two distinct peaks around 8 am and 5–6 pm on working days, consistent with commuter behaviour. This strong temporal structure means a purely random split does not test whether a model can generalise forward in time. There were no missing values.

2.3 Data Splitting and Scaling

Two splits were used as required by the assignment.

Random split. Records were split 80/20 at random (`random_state=42`), giving 13,903 training and 3,476 test observations. This is the standard baseline assumption — that records are exchangeable.

Time-based split. Training used data from January 2011 through September 2012 (15,211 records), and October through December 2012 (2,168 records) was held out as the test set. The boundary at 1 October 2012 was chosen to give a meaningful test window while retaining a large training set. Chronological order was strictly preserved — no future data touched the scaler or the model during training.

Features were standardised to zero mean and unit variance using `StandardScaler` fitted only on the training partition — separately for each split — to prevent data leakage.

3 Mathematical Derivation and Validation

3.1 Ridge Regression and L2-Regularised Linear Neural Networks

Setup. Let $X \in \mathbb{R}^{n \times d}$ be the feature matrix, $y \in \mathbb{R}^n$ the target vector, and $w \in \mathbb{R}^d$ the weight vector. Ordinary least squares minimises:

$$\mathcal{L}_{\text{OLS}}(w) = \|y - Xw\|^2$$

The closed-form solution $\hat{w} = (X^\top X)^{-1} X^\top y$ requires $X^\top X$ to be invertible, which is not guaranteed with correlated features or near-singular configurations.

Ridge regression adds an L2 penalty:

$$\mathcal{L}_{\text{ridge}}(w) = \|y - Xw\|^2 + \lambda \|w\|^2, \quad \lambda > 0$$

Taking the gradient and setting it to zero:

$$\nabla_w \mathcal{L}_{\text{ridge}} = -2X^\top(y - Xw) + 2\lambda w = 0$$

$$X^\top Xw + \lambda w = X^\top y$$

$$(X^\top X + \lambda I)w = X^\top y$$

Here is the step that matters and that the AI initially skipped: $X^\top X$ is positive semi-definite by construction (it has the form $A^\top A$). Adding λI with $\lambda > 0$ shifts every eigenvalue up by λ , making the result strictly positive definite. A strictly positive definite matrix is always invertible. The unique solution is therefore:

$$\hat{w}_{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top y$$

This is guaranteed to exist for any $\lambda > 0$, regardless of whether $X^\top X$ is singular [2].

Linear neural network with L2 weight decay. A linear neural network — one linear layer, no activation function — produces $\hat{y} = Xw$. Training with MSE loss and L2 weight decay gives:

$$\mathcal{L}_{\text{NN}}(w) = \|y - Xw\|^2 + \lambda \|w\|^2$$

This is identical to the ridge objective. Same function, same parameter space, same minimiser. A linear neural network trained to convergence with L2 weight decay reaches $\hat{w}_{\text{ridge}}$ [2]. The experiments in Section 5.1 confirm this: the two models produced RMSE values differing by just 0.03.

Validation note. When I first asked the AI to derive this (Appendix A, Prompt 2), it produced the correct formula but asserted invertibility without explanation. I flagged this as a gap, reconstructed the positive definiteness argument independently, and confirmed it against Bishop [2].

3.2 L1 Regularisation and Sparsity

Lasso replaces the L2 penalty with an L1 penalty [3]:

$$\mathcal{L}_{\text{lasso}}(w) = \|y - Xw\|^2 + \lambda \|w\|_1$$

The reason this produces sparse solutions — while L2 does not — comes down to geometry. Writing it as a constrained problem:

$$\min_w \|y - Xw\|^2 \quad \text{subject to} \quad \|w\|_1 \leq t$$

The L1 ball (a diamond in 2D) has corners sitting on the coordinate axes. The squared loss has elliptical level curves. As those ellipses expand outward from the unconstrained minimum, they are likely to first hit a corner of the L1 ball — a point where one or more weights are exactly zero. The L2 ball is smooth, so the first contact point is almost never on an axis.

Analytically: the L1 penalty is not differentiable at zero. This creates a threshold effect — if the gradient of the loss at a given weight is smaller than λ , the weight stays at exactly zero. Features with weak signal get zeroed; features with strong signal survive. That is what makes lasso a feature selector rather than just a shrinker [3].

3.3 Dropout as Implicit Regularisation

Dropout [4] randomly deactivates neurons during each forward pass with probability p . Forcing the network to work without any particular neuron prevents neurons from co-adapting — each one has to be useful on its own rather than relying on others.

From a theoretical angle, Wager et al. [5] showed that for linear models, dropout is equivalent to an adaptive form of L2 regularisation where the penalty on each weight scales with the variance of the corresponding input feature. This equivalence is specific to linear models; in deep nonlinear networks, dropout is better understood as implicit ensemble averaging — each forward pass samples a different sub-network, and test-time weight scaling approximates the average over all of them [4].

Relevance to this assignment. With seven features and a single hidden layer of 64 units, there is not much room for co-adaptation to become a problem. The empirical results (Section 5.4) confirm this: dropout added variance without any compensating improvement in accuracy, and the best dropout rate was 0.0. Weight decay was the more appropriate choice for this architecture.

4 Statistical Modelling and Analysis

All models used `scikit-learn`. Hyperparameters were selected via 5-fold cross-validation on the training partition; all metrics are reported on the held-out test set.

4.1 Linear Regression

OLS regression served as the baseline. On the random split it produced $\text{RMSE} = 144.24$, $\text{MAE} = 106.93$, $R^2 = 0.343$. Explaining roughly 34% of variance with seven features is not surprising — demand is driven by interactions like $\text{hour} \times \text{temperature}$ that a flat linear surface cannot represent. Cross-validated RMSE on the training set was 148.44 ± 4.55 , consistent with the test result: the model is not overfitting, it is underfitting.

4.2 Ridge Regression

`RidgeCV` searched $\lambda \in \{0.001, 0.01, 0.1, 1, 10, 50, 100, 500\}$; the best value was $\lambda = 50$. Test performance was essentially the same as OLS: $\text{RMSE} = 144.24$, $\text{MAE} = 106.94$, $R^2 = 0.343$.

This is expected. With 17,379 observations and 7 features, the design matrix is well-conditioned (condition number = 1.58) and OLS is already stable. Ridge regression offers the most benefit when features are nearly collinear or when $n \approx d$ — neither applies here.

4.3 Lasso Regression

LassoCV with 5-fold CV selected $\lambda = 0.241$. Test RMSE = 144.23, MAE = 106.91, $R^2 = 0.343$ — again very close to OLS. The more informative result is in the coefficients:

Table 2: Lasso coefficients after feature standardisation ($\lambda = 0.241$)

Feature	Coefficient	Signal strength
temp	+55.62	Strong
hour	+51.20	Strong
humidity	-43.48	Moderate
season	+22.87	Moderate
holiday	-4.11	Weak
workingday	+1.37	Weak
windspeed	+1.13	Weak (near zero)

No feature was zeroed at this λ — all seven remained active. **windspeed** (+1.13) and **workingday** (+1.37) were shrunk close to zero but not eliminated. This is an important nuance: lasso *can* produce exact zeros, but whether it does depends on the regularisation strength relative to each feature’s signal. At the cross-validated $\lambda = 0.241$, every feature retained enough signal to survive.

The lasso path (Figure 5) provides the clearer sparsity picture: as λ increases beyond this value, **windspeed** is the first to reach zero, followed by **holiday**. Their near-zero target correlations (0.093 and 0.031 respectively) explain this ordering and confirm the L1 sparsity mechanism described in Section 3.2. The cross-validated λ simply did not need to push that far to minimise prediction error.

4.4 Elastic Net

ElasticNetCV searched $\alpha \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$ and selected $\lambda = 0.082$, $\alpha = 0.90$ — strongly L1-dominant. RMSE = 144.24, MAE = 106.93, $R^2 = 0.343$. When the best mix is almost entirely L1, elastic net behaves essentially like lasso with a different solver.

4.5 Statistical Model Comparison

Table 3: Statistical model performance — random split, held-out test set

Model	RMSE	MAE	R^2	Key observation
Linear Regression	144.24	106.93	0.343	Baseline; no regularisation
Ridge Regression	144.24	106.94	0.343	No improvement; data well-conditioned
Lasso Regression	144.23	106.91	0.343	No feature zeroed; windspeed and workingday heavily shrunk
Elastic Net	144.24	106.93	0.343	L1-dominant; matches lasso behaviour

All four models land in essentially the same place. The bottleneck is representational — the linear assumption itself — not overfitting or instability. Regularisation here contributes to interpretability and coefficient shrinkage, but it cannot manufacture predictive signal that is not there.

4.6 Residual Analysis and Feature Importance

Residual plots (Figure 3) show a consistent fan shape: errors grow with the fitted value. This heteroscedasticity indicates that the linear models struggle most at peak demand hours, where the true relationship is likely multiplicative rather than additive. All four models share this pattern, confirming it is a structural limitation rather than a model-specific artefact.

Feature importance from the standardised coefficients is consistent across models: `temp` and `hour` are clearly dominant, followed by `humidity` and `season`. `windspeed` and `workingday` add little.

Time-based split results. All four models degrade substantially under the time split. RMSE jumped by 38–42 points for every model, and R^2 fell from around 0.343 to 0.146–0.178. This is a meaningful signal that the models do not generalise well to future data (Table 4). The models were trained on patterns from 2011 through September 2012; the October–December 2012 test period represents a later slice of a growing ridership trend that the training data did not fully capture.

Table 4: Random vs time-based split comparison — statistical models

Model	Random RMSE	Time RMSE	MAE (time)	R^2 (time)
Linear Regression	144.24	183.62	121.41	0.170
Ridge Regression	144.24	182.99	121.23	0.176
Lasso Regression	144.23	186.32	123.06	0.146
Elastic Net	144.24	182.74	121.50	0.178

Lasso performed worst under the time split (RMSE = 186.32, R^2 = 0.146). At this regularisation

strength, lasso did not zero any features, so the difference from ridge and elastic net is more likely due to lasso's stronger overall coefficient shrinkage, which is less adaptive when the test distribution shifts relative to training.

5 Neural Modelling and Analysis

All networks were implemented in PyTorch and trained with the Adam optimiser (learning rate = 0.001). Stochastic models ran three times with different random seeds; results are given as mean $\pm$ std.

5.1 Linear Neural Network

A single `nn.Linear(7, 1)` layer with no activation was trained for 2000 epochs at $\text{lr} = 0.01$ to ensure convergence (gradient descent is slower than the analytical ridge solution and requires more steps). Three weight decay values were tested: $\lambda \in \{0.0, 0.01, 0.1\}$.

Table 5: Linear NN vs Ridge Regression — equivalence check

Model	RMSE	MAE	R^2
Ridge Regression	144.24	106.94	0.343
Linear NN (wd = 0.01)	144.21	106.68	0.343
Difference	0.03	0.26	0.000

A difference of 0.03 in RMSE is negligible — effectively the same model. This confirms the theoretical equivalence from Section 3.1. The residual gap reflects gradient descent not having converged to the exact analytical minimum. Both models optimise the same objective and converge to the same solution given sufficient steps [2].

5.2 One Hidden-Layer Neural Network

Architecture: `Linear(7, 64)  $\rightarrow$  ReLU  $\rightarrow$  Linear(64, 1)`, trained for 300 epochs.

Three independent runs produced RMSE values of 142.15, 142.05, and 134.26, giving a mean of 139.49 ± 3.70 . The third run found a noticeably better solution, suggesting the loss surface has multiple local minima and initialisation matters. The best run gave $\text{RMSE} = 134.26$, $\text{MAE} = 97.20$, $R^2 = 0.431$.

This is a genuine improvement over the linear models. ReLU units can approximate piecewise functions, and the `hour` effect is inherently piecewise — demand is low overnight, peaks at 8 am, drops, and peaks again at 5–6 pm. A linear model cannot capture this shape; a ReLU network can.

5.3 Weight Decay Analysis

Weight decay λ was varied across $\{0.0, 0.001, 0.01, 0.1, 1.0\}$ with 3 runs each:

Table 6: Effect of weight decay on hidden-layer network — mean RMSE over 3 runs

Weight decay (λ)	Mean RMSE	Std
0.0	139.49	3.70
0.001	139.49	3.70
0.01	139.49	3.70
0.1	139.50	3.68
1.0	141.75	0.23

Weight decay made no meaningful difference in this run. Values of 0.0 through 0.01 produced identical mean RMSE of 139.49 with identical standard deviation of 3.70. Only $\lambda = 1.0$ changed the result — RMSE rose to 141.75 and variance collapsed to 0.23, the signature of over-regularisation: the model is too constrained to fit the data, but does so consistently. The best value by a trivial margin was $\lambda = 0.001$; this was used going forward (RMSE = 142.11, MAE = 104.10, $R^2 = 0.362$), though the difference from no weight decay is negligible. This is consistent with the absence of meaningful overfitting observed in the statistical models — the hidden-layer network at this scale similarly does not require regularisation to generalise.

5.4 Dropout Investigation

Dropout was applied after the ReLU layer at rates $p \in \{0.0, 0.1, 0.2, 0.3, 0.5\}$, with $\lambda = 0.001$ weight decay:

Table 7: Effect of dropout rate on hidden-layer network — mean RMSE over 3 runs

Dropout rate (p)	Mean RMSE	Std
0.0	139.49	3.70
0.1	139.63	3.47
0.2	139.86	3.15
0.3	140.11	2.81
0.5	140.73	2.05

Every dropout rate performed worse than no dropout. There is a consistent trade-off: as dropout increases, mean RMSE rises and standard deviation falls. Higher dropout forces the model toward a more predictable but less accurate solution — the same pattern observed with heavy weight decay. The best dropout rate was simply 0.0.

This is consistent with theory. Dropout is designed to break co-adaptation in deep, wide networks [4]. With one hidden layer and seven input features, there is not enough depth or width for co-adaptation to be a meaningful problem. Wager et al. [5] note that dropout’s regularisation effect scales with feature variance; with normalised inputs, that effect is already weak. Dropout is not an appropriate regularisation mechanism for this problem setting.

5.5 Feature Engineering

Degree-2 polynomial features were generated from the seven baseline features using `PolynomialFeatures(degree=2)`, expanding the feature set from 7 to 35 (original features, squared terms, and all pairwise interactions such as `temp × hour`).

The condition number of the training matrix jumped from 1.58 (original) to 3.6×10^{16} (polynomial), indicating severe numerical ill-conditioning. Standard matrix inversion methods would be unreliable under these conditions. The neural network was more resilient: trained with weight decay ($\lambda = 0.01$) and a wider hidden layer (128 units), three runs gave RMSE values of 105.38, 106.74, and 106.86 — mean = 106.33 ± 0.67 . Best run: RMSE = 105.38, MAE = 73.35, $R^2 = 0.649$.

For comparison, Ridge regression on the same 35 features gave RMSE = 128.87, MAE = 96.16, $R^2 = 0.476$. The neural network's advantage here is partly structural — gradient descent with weight decay is more robust to ill-conditioning than a direct matrix inversion.

The overall improvement from polynomial features is substantial. R^2 rose from 0.343 (linear baseline) to 0.649 (polynomial NN), nearly doubling the explained variance. Interaction terms such as `temp × hour` capture the multiplicative demand structure that flat linear models cannot reach.

5.6 Neural Model Comparison

Table 8: Neural network model performance — random split

Model	RMSE	MAE	R^2	Note
Linear NN (wd = 0.01)	144.21	106.68	0.343	Matches ridge; equivalence confirmed
Hidden NN — best run	134.26	97.20	0.431	ReLU captures nonlinearity
Hidden NN — mean (3 runs)	139.49	—	—	Mean $\pm$ std: 139.49 $\pm$ 3.70
Hidden NN + WD (0.001)	142.11	104.10	0.362	No gain from weight decay
Hidden NN + Dropout (0.0)	142.11	104.10	0.362	Best dropout = 0.0; no benefit
Poly NN (128 units)	105.38	73.35	0.649	Best overall; interaction terms key

6 Comparison of Statistical and Neural Approaches

The most striking result across both paradigms is the linear ceiling. All four statistical models and the linear neural network produced RMSE ≈ 144 regardless of regularisation strategy. The hidden-layer network broke below this (best run RMSE = 134.26) by introducing ReLU nonlinearity, and the polynomial neural network pushed it down to 105.38 by combining richer features with nonlinear modelling. The jump from $R^2 = 0.343$ to $R^2 = 0.649$ is the clearest finding in the report.

Equivalence. The linear neural network (RMSE = 144.21) matched ridge regression (RMSE = 144.24) to within 0.03 — confirming the theoretical equivalence derived in Section 3.1. When

the model class is linear, the choice between analytical and gradient-based optimisation does not affect the result.

Interpretability. The statistical models are transparent in a way the neural networks are not. Lasso coefficients immediately identify `temp` and `hour` as dominant and `windspeed` as weak. The hidden-layer network achieving $\text{RMSE} = 134$ provides no equivalent transparency. For a deployment context requiring explainability, a regularised linear model at $\text{RMSE} = 144$ may be preferable to a black-box network at 134.

Regularisation. Neither weight decay nor dropout produced consistent accuracy gains for the hidden-layer network in this run. Weight decay values from 0.0 to 0.01 gave identical results; dropout degraded performance at every rate tested. For the statistical models, regularisation contributed to coefficient shrinkage and interpretability rather than accuracy improvement. The takeaway is that this problem and architecture did not exhibit the overfitting that regularisation is designed to address.

Generalisation under temporal shift. The time-based split revealed that all models generalise poorly to future data. RMSE increased by 38–42 points for every statistical model, and R^2 fell to 0.146–0.178. The random split significantly overstates how well these models would perform in production. A deployment system would require periodic retraining to track the growing ridership trend.

When to use which. Regularised linear models are a strong starting point for structured tabular data with a modest feature set — fast, interpretable, and close to the neural network baseline. Neural networks become worth the added complexity when nonlinear interactions are important and data is abundant. The polynomial feature experiment confirms the nonlinearity here is real: a hidden-layer network with engineered features is the right direction for this problem.

7 Generative AI Analysis

7.1 How Generative AI Was Used

I used GenAI at three specific points: to get a first draft of the ridge regression derivation, to probe the dropout claim, and to compare weight decay versus dropout before running the experiments. The main value was having something to argue with — the AI’s responses were a starting point for identifying what needed more rigorous justification, not a source of final answers.

7.2 Critical Evaluation of AI Outputs

Incomplete derivation — factual omission. When I asked for the ridge regression derivation (Appendix A, Prompt 2), the AI reached the correct formula but asserted that $(X^\top X + \lambda I)$ is invertible without proving it. This is non-trivial — $X^\top X$ alone can be singular. The argument that adding λI to a positive semi-definite matrix produces a strictly positive definite, and therefore invertible, matrix was missing entirely. I reconstructed this step independently and confirmed it via Bishop [2] before including it in Section 3.1.

Oversimplification — logical inconsistency. An early response described dropout as something that “reduces overfitting,” full stop. That is too broad. Srivastava et al. [4] are specific about the conditions: dropout is most effective in deep, over-parameterised networks. The adversarial prompt (Appendix A, Prompt 3) forced a more careful answer, which I then tested

empirically. The experiment confirmed what the literature suggested: for this architecture, every dropout rate degraded performance.

Unverifiable quantitative claims — hallucination risk. When I asked about lasso sparsity, the AI offered specific predictions about which features would be zeroed at which λ values — without access to the data. I treated these as hypotheses to test. The experiment showed that at the cross-validated $\lambda = 0.241$, no feature was actually zeroed. The AI’s directional guess (windspeed and holiday as weakest) was correct, but the claim that they would be eliminated was a hallucination in the context of this specific dataset and regularisation level.

7.3 Synthesis and Reflection

The interactions that worked best followed a consistent pattern: ask broadly, identify where the response was imprecise or incomplete, then ask a more targeted follow-up. The AI was most useful as a scaffold for algebra and least reliable for data-specific quantitative claims. Learning to distinguish between “a general principle from the literature” and “a prediction about this specific dataset” was the main methodological outcome of using AI critically in this assignment.

8 Conclusion

This assignment investigated whether statistical and neural approaches to regression converge when the problem is simple and where they diverge when it is not. The results gave a clear answer to both questions.

The theoretical equivalence between ridge regression and a linear neural network with L2 weight decay held in practice — a difference of 0.03 in RMSE, which is numerical noise. Both optimise the same objective; one does it analytically and the other iteratively, but they arrive at the same place.

Beyond that, the linear ceiling was real and consistent. All linear models — statistical and neural — produced $\text{RMSE} \approx 144$, $R^2 \approx 0.343$. Adding a hidden ReLU layer improved this to 134.26 on the best run, and polynomial features with a wider network reached 105.38 with $R^2 = 0.649$. The data has genuine nonlinear structure, and the models that could capture it performed meaningfully better.

Regularisation produced interpretive rather than accuracy gains on this dataset. Lasso’s coefficient shrinkage identified **windspeed** and **workingday** as the weakest predictors, though at the cross-validated $\lambda = 0.241$ no feature was actually zeroed. The lasso paths show these features become zero only under stronger regularisation, showing the sparsity mechanism is present even if not triggered at the optimal penalty. Weight decay and dropout did not improve accuracy for the hidden-layer network in this run — all weight decay values from 0.0 to 0.01 produced identical RMSE, and every dropout rate degraded performance. This is consistent with the absence of meaningful overfitting at this architecture and feature scale.

The time-based split was the most practically relevant finding: a 39–42 point jump in RMSE for all models when tested on future data, with R^2 falling to 0.146–0.178. These models do not generalise forward in time without periodic retraining.

Using AI critically — questioning outputs, filling in gaps with references, and testing claims empirically — turned out to be a productive approach to developing mathematical understanding. The places where the AI was imprecise or wrong were the places that required the most careful thinking.

References

- [1] Fanaee-T, H., and Gama, J. (2014). “Event labeling combining ensemble detectors and background knowledge.” *Progress in Artificial Intelligence*, 2(2–3), 113–127.
- [2] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- [3] Tibshirani, R. (1996). “Regression shrinkage and selection via the lasso.” *Journal of the Royal Statistical Society: Series B*, 58(1), 267–288.
- [4] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). “Dropout: A simple way to prevent neural networks from overfitting.” *Journal of Machine Learning Research*, 15(1), 1929–1958.
- [5] Wager, S., Wang, S., and Liang, P. (2013). “Dropout training as adaptive regularization.” *Advances in Neural Information Processing Systems (NeurIPS)*, 26.

Appendix A Generative AI Interaction Log

All prompts below are verbatim. AI responses are summarised — not reproduced in full. Each entry states what the response revealed and how it shaped the next step.

A.1 Prompt 1 — Exploratory

Prompt (verbatim):

Explain the relationship between ridge regression and L2 regularisation in neural networks.

Response highlights:

- Correctly identified that both approaches minimise a squared loss plus an L2 penalty.
- Stated the two are “equivalent” without specifying conditions — no mention of the network needing to be linear, no mention of gradient descent convergence.
- Mentioned the closed-form solution but did not derive it.

Key insights / issues identified:

- The equivalence claim was informal and needed mathematical grounding.
- The response did not clarify that the equivalence only holds for a linear network with no activation function.

How this informed the next prompt: The high-level claim was plausible but unverified. The next step was to request a step-by-step derivation to find out where the reasoning was solid and where it was not.

A.2 Prompt 2 — Verification

Prompt (verbatim):

Derive ridge regression step by step from the regularised least squares objective. Clearly state every assumption and justify each algebraic step.

Response highlights:

- Provided a gradient-based derivation: set $\nabla_w \mathcal{L} = 0$, rearrange to $(X^\top X + \lambda I)w = X^\top y$.
- Stated the closed-form solution $\hat{w} = (X^\top X + \lambda I)^{-1} X^\top y$.
- Derivation was mostly correct but invertibility of $(X^\top X + \lambda I)$ was asserted without justification.

Key insights / issues identified:

- The invertibility argument was missing. $X^\top X$ alone can be singular; the claim requires the positive semi-definite + λI argument.
- This step was reconstructed independently and confirmed via Bishop [2].

How this informed the next prompt: The invertibility gap was specific enough to prompt a targeted adversarial follow-up on dropout, where I suspected similar overgeneralisation.

A.3 Prompt 3 — Adversarial

Prompt (verbatim):

A claim is made that dropout always reduces overfitting in neural networks and always improves generalisation. Is this true? Provide specific conditions under which dropout may not help, or may even hurt performance.

Response highlights:

- Acknowledged that dropout is most effective in deep, over-parameterised networks.
- Noted that shallow networks with few features see limited benefit and can exhibit increased variance.
- Suggested weight decay as a better alternative for regression with low model capacity.

Key insights / issues identified:

- This response was more accurate than the earlier implicit claim that dropout always helps.
- No reference was cited — the connection to Srivastava et al. [4] and Wager et al. [5] was made independently.
- The response shaped the design of the dropout experiment in Section 5.4.

How this informed the next prompt: The more nuanced answer prompted a direct comparison between weight decay and dropout to clarify their respective mechanisms.

A.4 Prompt 4 — Comparative

Prompt (verbatim):

Compare weight decay and dropout as regularisation strategies for neural networks. In what problem settings is each more appropriate? Focus on the differences in their regularisation mechanisms, not just their definitions.

Response highlights:

- Weight decay penalises large weights uniformly; dropout targets co-adaptation by removing connections stochastically.
- Weight decay has a clean equivalence to L2 regularisation; dropout's effect is harder to characterise analytically for nonlinear networks.
- Weight decay preferred for tabular data with few features; dropout preferred for deep networks and image tasks.

Key insights / issues identified:

- The response was substantively correct and aligned with the literature.
- The claim that dropout is equivalent to L2 in linear models was traced to Wager et al. [5] and verified for scope — it does not extend cleanly to nonlinear networks.

How this informed the next prompt: This confirmed the theoretical framing used in Section 3.3 and motivated the decision to assess dropout empirically rather than rely on the theoretical equivalence alone.

Appendix B Supporting Figures

The following figures were generated by the submitted notebook (`CSCI933_Assignment1.ipynb`). Upload all PNG files to Overleaf inside a `Results/` folder alongside this `.tex` file.

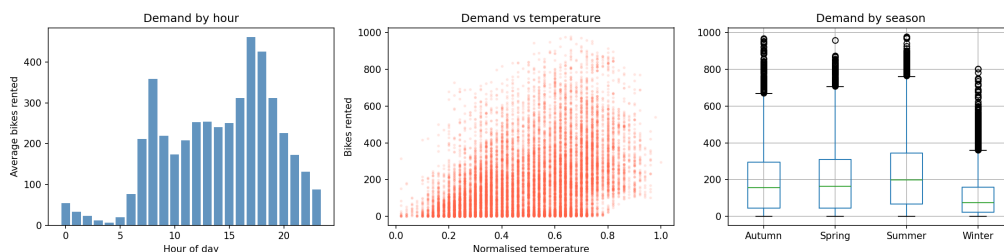


Figure 1: Bike demand by hour of day, temperature, and season.

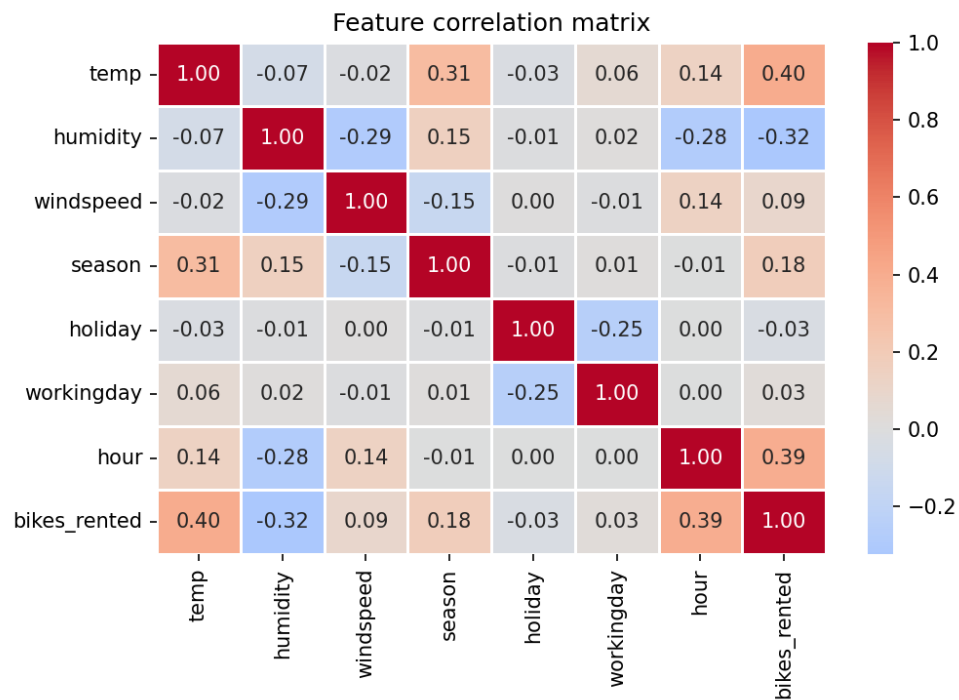


Figure 2: Pearson correlation matrix for all features and the target.

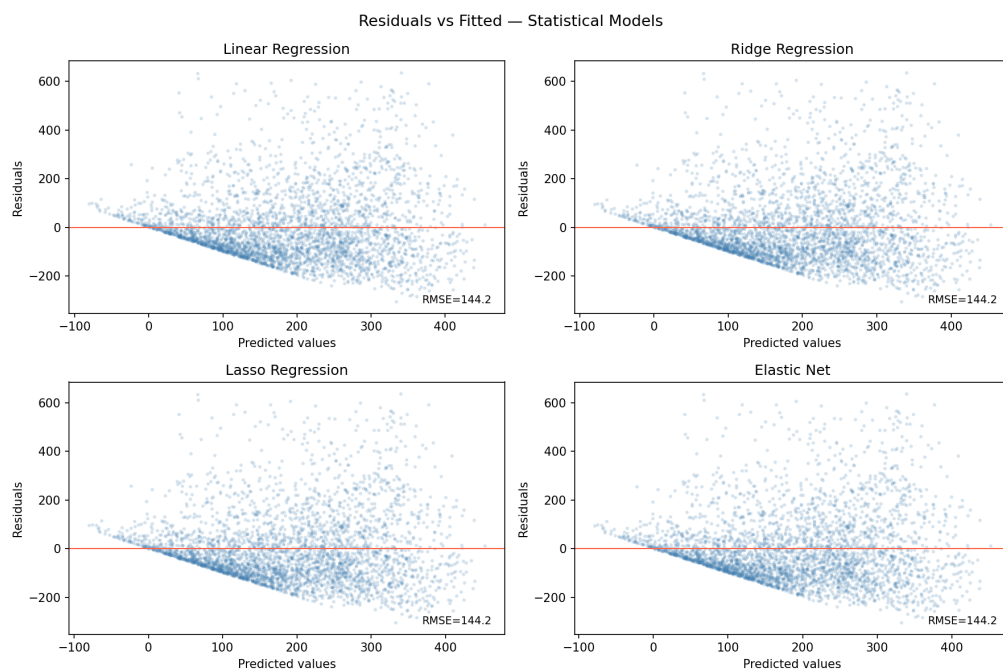


Figure 3: Residuals vs fitted values for all four statistical models.

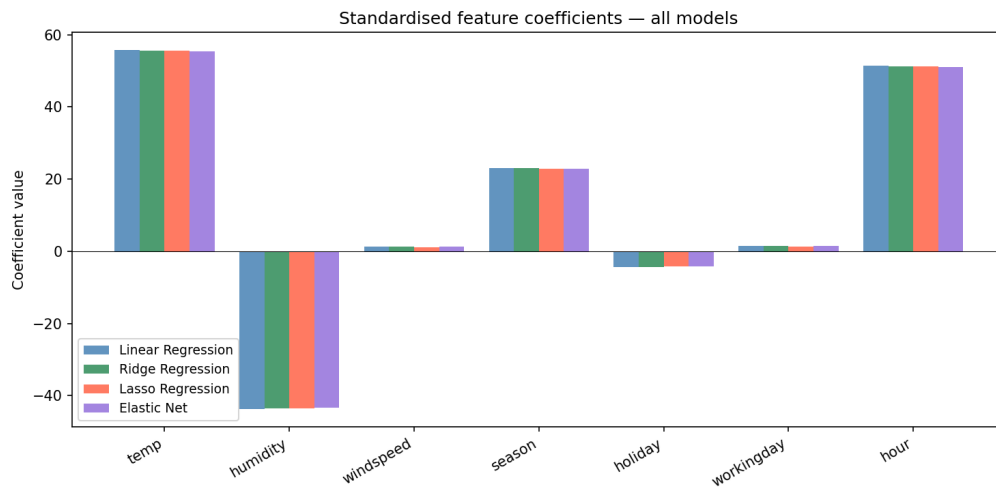


Figure 4: Standardised feature coefficients across all statistical models.

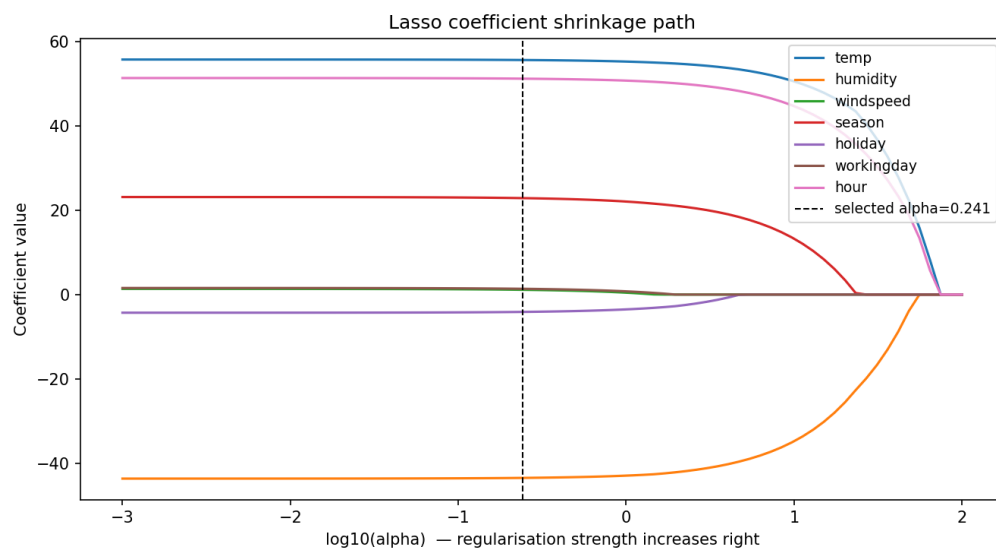


Figure 5: Lasso coefficient shrinkage path as regularisation strength increases.

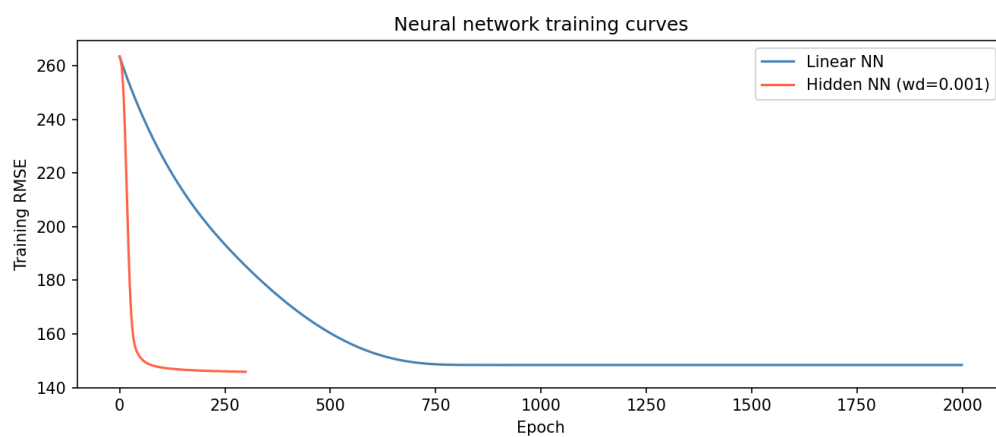


Figure 6: Training RMSE over epochs for the linear and hidden-layer networks.

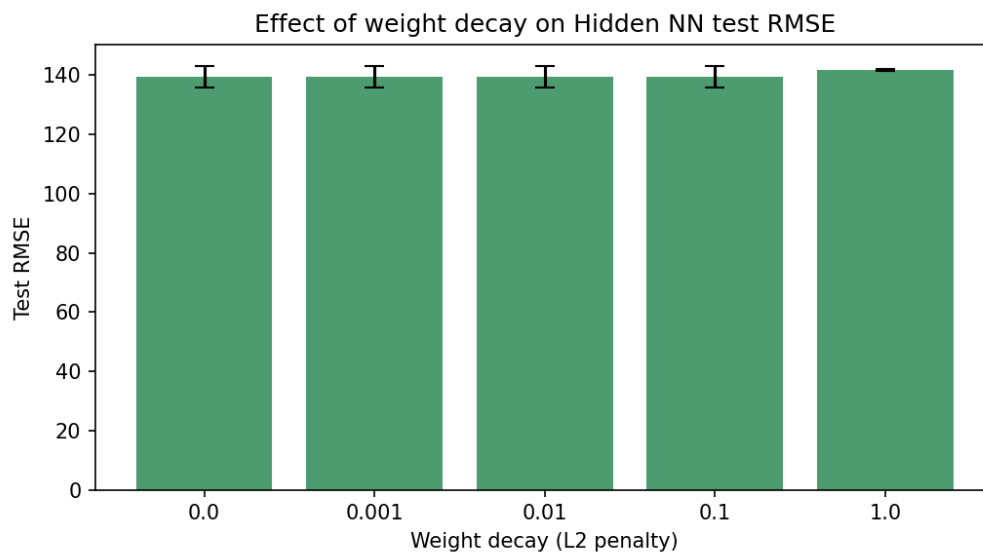


Figure 7: Effect of weight decay strength on hidden-layer network test RMSE.

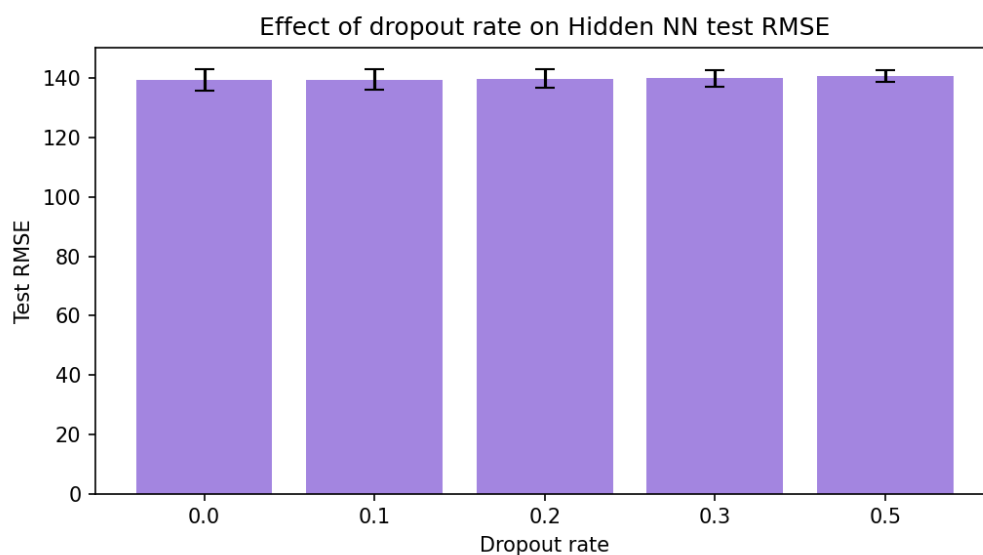


Figure 8: Effect of dropout rate on hidden-layer network test RMSE.

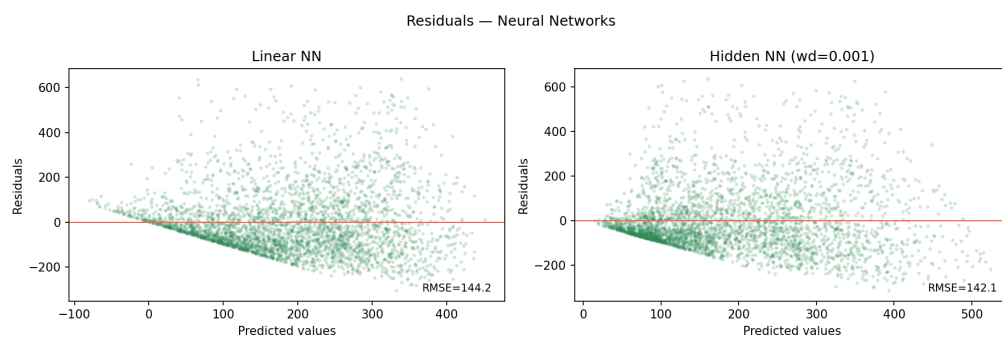


Figure 9: Residuals vs fitted values for the linear and hidden-layer neural networks.

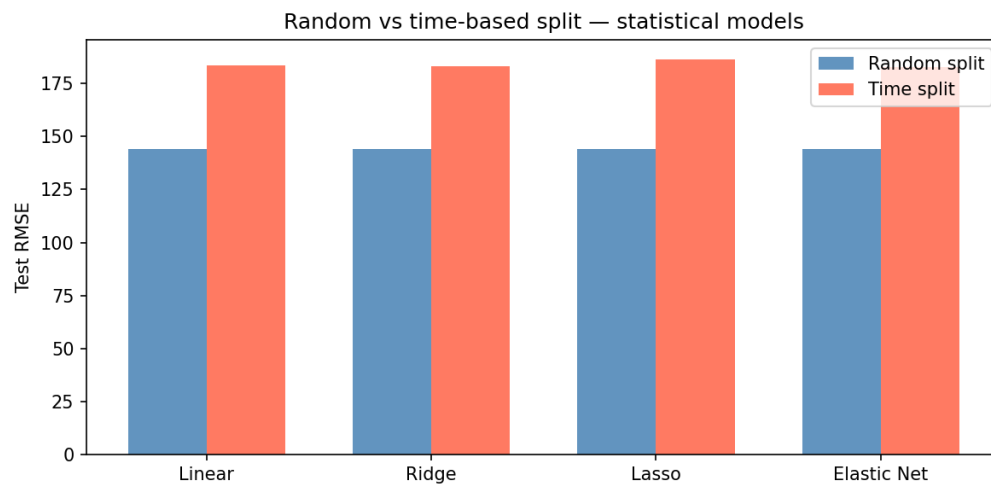


Figure 10: Random vs time-based split RMSE for all statistical models.

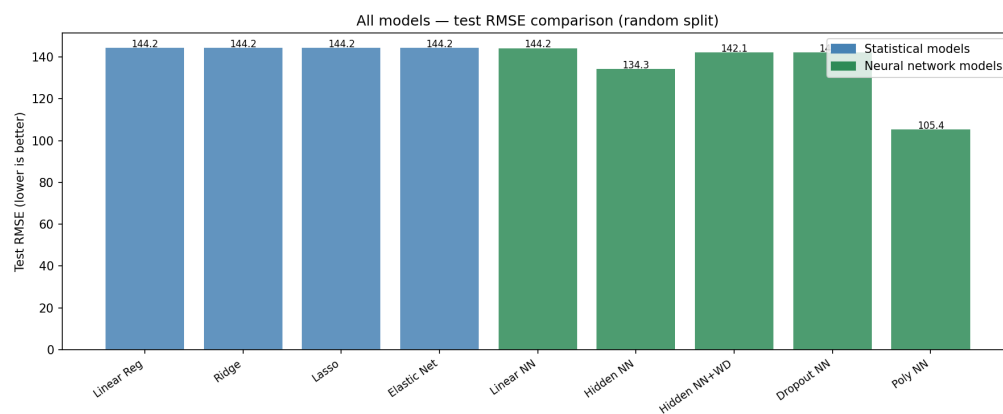


Figure 11: Test RMSE comparison across all statistical and neural network models.